LEHR- UND PRAXISLEITFADEN · PROF. DR. MATHIAS ENGEL

# Trinity Agent Architektur:
## *Autarker Praxisleitfaden für Studierende*

Vom monolithischen Prompt zum deterministischen, vertragsbasierten Multi-File-Agenten mit OpenAI-/Codex-Harness, SQLite Memory und transparenter WebUI mit Datei-Inspektor.

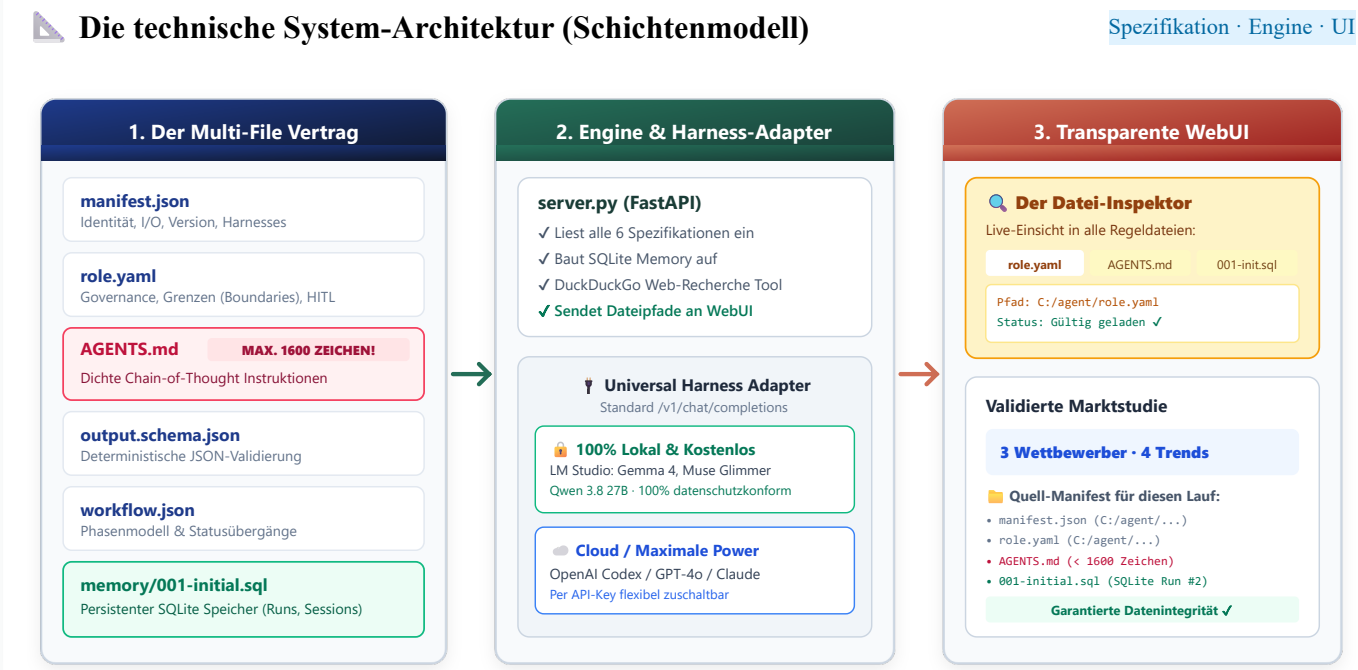
🛠️ **Die Metapher: Der Handwerker-Agent**

Didaktisches Konzeptbild




**Die Handwerker-Metapher auf einen Blick:** Ein echter KI-Agent handelt wie ein Meister in seiner Werkstatt:

- **Der Auftrag & die Leitplanken (role.yaml):** Der Bauplan auf der Werkbank definiert unüberwindbare Schranken und Qualitätskriterien.
- **Das Fachwissen (AGENTS.md):** Ein kompaktes Handbuch (≤ 1600 Zeichen) mit klaren Chain-of-Thought-Arbeitsschritten.
- **Die Werkzeuge (manifest.json & Harness):** Die Werkzeugwand mit Messinstrumenten und Schnittstellen (z. B. DuckDuckGo, LLM-Harness).
- **Das Werkstatt-Gedächtnis (001-initial.sql):** Schubladen und Archivkästen für persistente SQLite-Speicherung früherer Recherchen.
- **Das fertige Werkstück (output.schema.json):** Ein präzise kalibriertes, maschinenlesbares Ergebnis ohne Formfehler.




## 1. Einleitung: Was entsteht hier und was macht diesen Ansatz besonders?

Klassische KI-Anwendungen im Studium beginnen meist mit einem langen Chat-Prompt in Web-Interfaces wie ChatGPT oder Claude. Für einfache Fragen genügt das. Sobald jedoch reale, automatisierbare Geschäftsprozesse abgebildet werden sollen, scheitern solche monolithischen „Mega-Prompts“.

> 💡 **Das didaktische Kernprinzip für Studierende:**
> Mit diesem Leitfaden baust du keinen simplen Chatbot, sondern ein vollwertiges, industrietaugliches **KI-Agenten-System**. Das System läuft **100% autark** auf deinem eigenen Rechner (ohne Abhängigkeit von einem zentralen Server), schützt durch lokale LLM-Unterstützung vertrauliche Daten und beweist durch den integrierten **Datei-Inspektor** bei jeder Ausführung, aus welcher Datei welche Entscheidung abgeleitet wurde.

**Die beiden goldenen Architektur-Regeln**

**REGEL 1: SYSTEM-PROMPT SCHLANK HALTEN**

**Die Datei `AGENTS.md` darf maximal 1600 Zeichen umfassen!**

Ein häufiger Fehler von KI-Einsteigern sind riesige, unübersichtliche Systemprompts („Prompt-Spaghetti“). Im Trinity-Standard erzwingen wir maximale Prägnanz: Dichte Chain-of-Thought-Instruktionen, klare Kriterien und kein Fülltext. Alle Schranken gehören in `role.yaml`, die Datenstrukturen in `output.schema.json`!

**REGEL 2: PERSISTENTES GEDÄCHTNIS MIT SQL**

> **Jeder Agent besitzt ein SQLite-Gedächtnis via `memory/migrations/001-initial.sql`**
>
> Ein professioneller Agent darf nicht bei jedem Neustart alles vergessen. Die SQL-Migration definiert Tabellen für Sitzungen ( `sessions` ), Recherchen ( `research_reports` ) und Wettbewerbsdaten ( `competitor_intel` ). Das Backend initialisiert die Datenbank beim Start automatisch.

**Die 6 Säulen des Trinity-Dateivertrags**

| Datei | Format | Zweck & Bedeutung |
|---|---|---|
| `manifest.json` | JSON | **Visitenkarte & Metadaten:** ID, Name, Version, Inputs, Outputs, Memory-Pfad und erlaubte LLM-Harnesses. |
| `role.yaml` | YAML | **Governance & Schranken:** Persona, Leitbild, strikte Verbote (*Boundaries*), Qualitätsprüfungen (*Quality Gates*) und Mensch-im-Loop-Freigaben (*Human Gates*). |
| `AGENTS.md` | Markdown | **Operatives Handbuch:** Chain-of-Thought-Denkschritte & Richtlinien. **Strikt limitiert auf max. 1600 Zeichen!** |
| `output.schema.json` | JSON-Schema | **Garantierte Ausgabestruktur:** Erzwingt maschinenlesbare Pflichtfelder (Zusammenfassung, Wettbewerber, Trends, Chancen, Detailbericht). |
| `workflow.json` | JSON | **Phasenmodell:** Definiert die linearen Bearbeitungsphasen (z. B. Scope → Web-Recherche → Synthese). |
| `memory/migrations/001-initial.sql` | SQL (SQLite) | **Persistentes Gedächtnis:** Schema für Sitzungen, Rechercheberichte und Marktdaten. |

## 2. Der universelle Master-Prompt für KI-Coding-Assistenten

Kopiere den folgenden Textblock vollständig und füge ihn in ein KI-Entwicklungstool deiner Wahl ein (z. B. **Antigravity**, **OpenAI Codex / ChatGPT**, **Claude** oder **Cursor**). Die KI wird dir auf Knopfdruck das vollständige, lauffähige Projekt erstellen.

```
Du bist ein erfahrener KI-Architekt und sollst für mich einen vollständigen, autarken "Trinity Agenten" entwickeln.
Der Agent soll dem Trinity-Architekturstandard entsprechen, jedoch völlig unabhängig von einem zentralen Hub laufen. Er

### 1. Die didaktischen Kernprinzipien & harten Regeln
- Im Trinity-Standard ist ein Agent kein monolithischer Skriptcode, sondern ein definierter Vertrag aus mehreren modula
- HARTE ARCHITEKTUR-REGEL: Die Datei AGENTS.md darf MAXIMAL 1600 ZEICHEN umfassen! Halte sie prägnant (Chain-of-Thought
- SQLITE GEDÄCHTNIS: Der Agent besitzt ein persistentes SQLite-Schema unter memory/migrations/001-initial.sql für Sitzu
- DATEIPFAD-TRANSPARENZ: Sowohl in der WebUI als auch bei der Ausführung MUSS transparent angezeigt werden, welche Date

### 2. Projektstruktur
Erstelle folgendes Projektverzeichnis:
my-trinity-agent/
├── manifest.json                    # Metadaten, Fähigkeiten, I/O-Definition & Memory-Pfad
├── role.yaml                        # Persona, Purpose, Grenzen (Boundaries), Quality Gates & HITL
├── AGENTS.md                        # Detaillierte Systeminstruktionen (STRIKT MAX 1600 ZEICHEN!)
├── output.schema.json               # JSON-Schema für validierte, strukturierte Ausgaben
├── workflow.json                    # Phasenmodell des Agenten (z. B. Scope -> Recherche -> Synthese)
├── memory/
│   └── migrations/
│       └── 001-initial.sql          # SQLite-Schema für sessions, research_reports, competitor_intel
├── server.py                        # Autarker Server (FastAPI + SQLite) mit DuckDuckGo & OpenAI/Codex-Harness
└── static/
    └── index.html                   # Single-Page-WebUI mit Datei-Inspektor und Ausführungsmaske

### 3. Beispiel-Agent: "Marktrecherche & Trend-Analyst"
Nutze als konkreten Beispiel-Agenten einen unkritischen, praxisnahen Wirtschafts-Analysten:
- Zweck: Recherchiert Märkte, Wettbewerber, Branchentrends und Markteintrittsbarrieren für ein vorgegebenes Thema (z. B
- Boundaries: Keine spekulativen Zahlen ohne Quellhinweis; Trennung zwischen belegten Fakten und Markthypothesen; sachl
- Quality Gates: Mindestens 3 konkrete Wettbewerber identifizieren; mindestens 3 technologische/wirtschaftliche Trends

### 4. Das Backend (server.py mit FastAPI und SQLite)
Erstelle ein lauffähiges Python-Backend, das:
1. Beim Start die SQLite-Datenbank agent_memory.sqlite3 mit 001-initial.sql initialisiert.
2. Die lokalen Dateien beim Start einliest und deren absolute Dateipfade ermittelt.
3. Einen Endpoint GET /api/agent/files bereitstellt, der alle Dateien inklusive Pfaden und Inhalten an die WebUI liefer
4. Optional: Eine einfache Websuche (DuckDuckGo via duckduckgo-search oder httpx) zur Anreicherung aktueller Marktfakte
5. Einen Endpoint POST /api/agent/run bereitstellt, der:
```

```
   - Die Nutzereingabe (Marktthema) empfängt.
   - Den System-Prompt dynamisch aus role.yaml und AGENTS.md kompiliert.
   - Eine konfigurierbare OpenAI-/Codex-Schnittstelle aufruft (/v1/chat/completions – kompatibel mit OpenAI, LM Studio
   - Die Antwort gegen output.schema.json validiert.
   - Das Ergebnis dauerhaft in SQLite (sessions und research_reports) speichert.
   - Im Response-Objekt und in der Konsole explizit die Liste aller verwendeten Quell-Dateien und deren Pfade mit zurüc

### 5. Die WebUI (static/index.html)
Erstelle eine moderne, responsive Single-Page-Anwendung:
1. Header: Zeigt Agenten-Name, Version und Status aus manifest.json.
2. Harness-Konfiguration: Eingabefelder für Base-URL (z. B. http://localhost:1234/v1), Modellname und optional API-Key.
3. Datei-Inspektor: Tabs, die alle Agenten-Dateien (inkl. 001-initial.sql) anzeigen. Klickt man auf eine Datei, sieht m
4. Eingabebereich: Textfeld für das zu recherchierende Marktthema und ein "Markt analysieren"-Button.
5. Ergebnisanzeige: Zeigt Marktüberblick, Wettbewerber-Tabelle, Trends, Chancen, Detailbericht und einen Kasten "Verwen

Gib mir den kompletten Code für alle Dateien und eine kurze Startanleitung.
```

## 3. Die Steuerdateien des Referenz-Agenten im Detail

**A.** `manifest.json`

```
{
  "schemaVersion": "1.0",
  "id": "market-research-agent",
  "name": "Marktrecherche & Trend-Analyst",
  "shortName": "MarketResearcher",
  "version": "1.0.0",
  "description": "Autarker Trinity-Agent zur fundierten Recherche von Märkten, Wettbewerbern und Trends.",
  "category": "Marktanalyse & Strategie",
  "capabilities": [
    "Strukturierte Marktanalyse",
    "Wettbewerber-Identifikation",
    "Trend- & Chancen-Scouting",
    "Quellengestützte Marktübersicht"
  ],
  "harnesses": ["codex", "chatgpt", "lmstudio", "ollama"],
  "memory": {
    "schemaVersion": 1,
    "migration": "memory/migrations/001-initial.sql"
  },
  "specFiles": {
    "manifest": "manifest.json",
    "role": "role.yaml",
    "instructions": "AGENTS.md",
    "outputSchema": "output.schema.json",
    "workflow": "workflow.json",
    "memorySql": "memory/migrations/001-initial.sql"
  }
}
```

**B.** `role.yaml`

```
schema_version: 1
role: MarketResearcher
purpose: Systematische Marktrecherche, Wettbewerbsanalyse und Trendidentifikation für innovative Geschäftsfelder.
boundaries:
  - Keine spekulativen Marktzahlen ohne Angabe der Quellbasis.
  - Klare Trennung zwischen belegten Marktfakten und strategischen Hypothesen.
  - Neutraler, faktenbasierter Wirtschaftsanalysten-Ton; keine Werbesprache.
quality_gates:
  - Mindestens 3 konkrete Wettbewerber oder Marktteilnehmer identifizieren.
  - Mindestens 3 wesentliche Markttreiber oder Trends herausarbeiten.
  - Ableitung konkreter Marktchancen (Opportunities) und Markteintrittsbarrieren.
human_gates:
  - Freigabe der finalen Markteintrittsstrategie durch Projektleitung / Dozenten.
```

**C.** `AGENTS.md` **(Exakt 928 Zeichen – Limit: max. 1600!)**

```
# Marktrecherche & Trend-Analyst – System-Instruktionen

## Ziel
Führe eine strukturierte, faktenbasierte Marktrecherche für das angefragte Thema oder Geschäftsfeld durch.

## Vorgehen (Chain-of-Thought)
1. Marktabgrenzung: Definiere Zielmarkt, Kernzielgruppe und das gelöste Kundenproblem.
2. Recherche-Vektoren: Untersuche den Markt entlang von drei Achsen:
   - Wettbewerber: Wer bietet bereits ähnliche Lösungen an? Was sind deren Stärken/Schwächen?
   - Trends & Treiber: Welche technologischen, gesellschaftlichen oder regulatorischen Treiber prägen den Markt?
   - Chancen & Risiken: Wo liegen unbesetzte Nischen und Markteintrittsbarrieren?
3. Synthese: Führe die Erkenntnisse in einem prägnanten Management-Briefing zusammen.

## Qualitätskriterien
- Quellenbezug: Jede Marktaussage muss nachvollziehbar belegt oder als fundierte Hypothese gekennzeichnet sein.
- Konkretheit: Nenne reale Marktnamen, Technologien und messbare Trends statt vager Verallgemeinerungen.

## Ausgabeformat
Antworte AUSSCHLIESSLICH mit einem validen JSON-Objekt, das exakt dem output.schema.json entspricht. Keine Einleitungst
```

**D.** **`memory/migrations/001-initial.sql`**

```
-- Persistentes SQLite-Gedächtnis für Marktrecherchen
CREATE TABLE IF NOT EXISTS sessions (
  id TEXT PRIMARY KEY,
  title TEXT NOT NULL,
  created_at TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS research_reports (
  id TEXT PRIMARY KEY,
  session_id TEXT NOT NULL,
  market_topic TEXT NOT NULL,
  market_trend TEXT NOT NULL,
  report_json TEXT NOT NULL,
  created_at TEXT NOT NULL,
  FOREIGN KEY(session_id) REFERENCES sessions(id)
);

CREATE TABLE IF NOT EXISTS competitor_intel (
  id TEXT PRIMARY KEY,
  competitor_name TEXT NOT NULL,
  market_topic TEXT NOT NULL,
  intel_data TEXT NOT NULL,
  created_at TEXT NOT NULL
);
```

**E.** **`output.schema.json`**

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "properties": {
    "marketSummary": {
      "type": "string",
      "description": "Kompaktes Management Summary zum untersuchten Markt"
    },
    "marketSizeTrend": {
      "type": "string",
      "description": "Einschätzung zur Marktreife und Wachstumsdynamik (z. B. Emerging, Wachstumsphase, Reif)"
    },
    "competitors": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "name": { "type": "string" },
          "focus": { "type": "string" },
          "advantage": { "type": "string" }
        },
        "required": ["name", "focus", "advantage"]
```

```
      }
    },
    "keyTrends": {
      "type": "array",
      "items": { "type": "string" }
    },
    "opportunities": {
      "type": "array",
      "items": { "type": "string" }
    },
    "detailedAnalysis": {
      "type": "string",
      "description": "Ausführlicher Marktbericht im Markdown-Format inklusive Quellen & Suchvektoren"
    }
  },
  "required": ["marketSummary", "marketSizeTrend", "competitors", "keyTrends", "opportunities", "detailedAnalysis"]
}
```

### 4. Quickstart & Datenschutz mit lokalem LLM

1. **Pakete installieren:** `pip install fastapi uvicorn httpx jsonschema pyyaml pydantic duckduckgo-search`
2. **Server starten:** `python server.py` (erstellt automatisch die `agent_memory.sqlite3`)
3. **WebUI öffnen:** Rufe im Browser `http://127.0.0.1:8000` auf.

🔒 **Kostenlos & 100% Datenschutz mit LM Studio / Ollama:**
Für sensible Unternehmensrecherchen oder interne Projektideen: Starte **LM Studio**, lade ein aktuelles Spitzenmodell (z. B. *Gemma 4 26B*, *Gemma 4 12B* bzw. *Gemma 4 E4B*, *Muse Glimmer* oder *Qwen 3.8 27B*) und klicke auf „Start Server“ auf Port `1234`. In der WebUI des Agenten bleibt die Base-URL auf `http://localhost:1234/v1` aktiv. Die Daten verlassen niemals deinen Laptop!